Python Set Revision Chart

This revision sheet covers Python set properties, mutability, built-in functions, methods, and operations with simple examples and outputs.[cite:43][cite:53][cite:59][cite:61]

Set chart

Section	Number	Topic	Explanation	Example	Output
Properties	1	Syntax	Set is usually written with curly braces `{}`.[cite:61]	`s = {1, 2, 3}`	`{1, 2, 3}`
Properties	2	Empty set	Empty set is written as `set()`, not `{}`, because `{}` creates an empty dictionary. [cite:61]	`s = set()`	`set()`
Properties	3	Mutability	Set is mutable, which means elements can be added, updated, or removed after creation. [cite:59] [cite:61]	`s = {1, 2}` then `s.add(3)`	`{1, 2, 3}`
Properties	4	Unique values	Duplicate values are removed automatically. [cite:60] [cite:61]	`s = {1, 2, 2, 3}`	`{1, 2, 3}`
Properties	5	Unordered	Set does not keep fixed order of elements. [cite:61]	`{3, 1, 2}`	Order may vary
Properties	6	No indexing	Set elements cannot be accessed with index position like `s[0]`. [cite:61]	`s = {1,2,3}` `print(s[0])`	Error
Properties	7	No slicing	Set does not support slicing, so start, stop, and step do not work. [cite:61]	`s = {1,2,3}` `print(s[0:2])`	Error

Section	Number	Topic	Explanation	Example	Output
Properties	8	Membership testing	Set is very useful for checking whether a value exists. [cite:61]	`print(2 in {1, 2, 3})`	`True`
Built-in Function	1	`len()`	Returns total number of elements. [cite:43]	`print(len({1, 2, 3}))`	`3`
Built-in Function	2	`min()`	Returns smallest element. [cite:43]	`print(min({3, 1, 2}))`	`1`
Built-in Function	3	`max()`	Returns largest element. [cite:43]	`print(max({3, 1, 2}))`	`3`
Built-in Function	4	`sum()`	Returns sum of numeric elements. [cite:43]	`print(sum({1, 2, 3}))`	`6`
Built-in Function	5	`sorted()`	Returns a sorted list from the set. [cite:43]	`print(sorted({3, 1, 2}))`	`[1, 2, 3]`
Built-in Function	6	`in`	Checks whether an element exists in the set.[cite:43]	`print(2 in {1, 2, 3})`	`True`
Built-in Function	7	`not in`	Checks whether an element does not exist in the set. [cite:43]	`print(5 not in {1, 2, 3})`	`True`
Method	1	`add()`	Adds one element to the set. [cite:53] [cite:59]	`s = {1, 2}` `s.add(3)` `print(s)`	`{1, 2, 3}`
Method	2	`update()`	Adds multiple elements from another iterable. [cite:53]	`s = {1, 2}` `s.update([3,4])` `print(s)`	`{1, 2, 3, 4}`
Method	3	`remove()`	Removes an element; gives an error if the item is missing. [cite:53] [cite:59]	`s = {1, 2, 3}` `s.remove(2)` `print(s)`	`{1, 3}`

Section	Number	Topic	Explanation	Example	Output
Method	4	`discard()`	Removes an element; gives no error if the item is missing.[cite:53][cite:59]	`s = {1, 2, 3}` `s.discard(5)` `print(s)`	`{1, 2, 3}`
Method	5	`pop()`	Removes one arbitrary element.[cite:53][cite:59]	`s = {10, 20, 30}` `s.pop()`	Removes one item
Method	6	`clear()`	Removes all elements from the set.[cite:53][cite:59]	`s = {1, 2}` `s.clear()` `print(s)`	`set()`
Method	7	`copy()`	Creates a copy of the set.[cite:53][cite:59]	`a = {1,2}` `b = a.copy()` `print(b)`	`{1, 2}`
Method	8	`union()`	Returns all unique elements from both sets.[cite:53][cite:60]	`print({1,2}.union({2,3}))`	`{1, 2, 3}`
Method	9	`intersection()`	Returns common elements of sets.[cite:53][cite:60]	`print({1,2}.intersection({2,3}))`	`{2}`
Method	10	`difference()`	Returns elements in the first set but not in the second.[cite:53][cite:60]	`print({1,2,3}.difference({2}))`	`{1, 3}`
Method	11	`symmetric_difference()`	Returns elements that are in one set only, not in both.[cite:53]	`print({1,2}.symmetric_difference({2,3}))`	`{1, 3}`
Method	12	`issubset()`	Checks whether all elements of one set are inside another set.[cite:53][cite:59]	`print({1,2}.issubset({1,2,3}))`	`True`
Method	13	`issuperset()`	Checks whether one set contains all elements of another set.[cite:53][cite:59]	`print({1,2,3}.issuperset({1,2}))`	`True`

Section	Number	Topic	Explanation	Example	Output
Method	14	`isdisjoint()`	Checks whether two sets have no common elements.[cite:53][cite:59]	`print({1,2}.isdisjoint({3,4}))`	`True`
Operation	1	Union `|`	Joins all unique elements from both sets.[cite:53][cite:60]	`print({1,2} | {2,3})`	`{1, 2, 3}`
Operation	2	Intersection `&`	Gives common elements only.[cite:53][cite:60]	`print({1,2} & {2,3})`	`{2}`
Operation	3	Difference `-`	Gives elements in the first set but not in the second.[cite:53][cite:60]	`print({1,2,3} - {2})`	`{1, 3}`
Operation	4	Symmetric difference `^`	Gives elements that are different in both sets.[cite:53][cite:60]	`print({1,2} ^ {2,3})`	`{1, 3}`
Operation	5	Membership `in`	Checks whether an element is present.[cite:43][cite:61]	`print(3 in {1,2,3})`	`True`
Operation	6	Membership `not in`	Checks whether an element is absent.[cite:43][cite:61]	`print(7 not in {1,2,3})`	`True`

Start, stop, step note

Start, stop, and step belong to indexing and slicing syntax such as `data[start:stop:step]`, but this works for ordered sequence types like strings and tuples, not for sets because sets are unordered.[cite:61] For sets, the correct rule is that indexing and slicing are not supported.[cite:59][cite:61]

Mutability note

Set is mutable because methods like `add()`, `update()`, `remove()`, `discard()`, `pop()`, and `clear()` can change the set after it is created.[cite:53][cite:59][cite:61] Even though a set is mutable, it still does not support positional access like indexing or slicing.[cite:61]